HS Data Science 26

Session 03: Scientific Working II — Literature, Publishing & Project Management

Prof. Dr. Michael Granitzer

5 May 2026

Session Overview

Literature Research

- where and how to find relevant papers
- reading strategies at scale
- forward and backward citation tracking
- organising and managing references

Scientific Publishing

- anatomy of a research paper
- the peer review process
- where to publish in AI/ML/IR
- common rejection reasons

Project Management

- why researchers need a system
- GTD adapted for research work
- Git as a research infrastructure tool
- repository structure for ML experiments
- the research diary / lab notebook

> Good science requires good habits. This session is about the infrastructure around the research loop, not the loop itself.

Literature Research

Guiding Question: How do you find and map the papers that matter for your topic?

Literature Research — Overview

Module Overview

You cannot find a gap without knowing the field. Literature research is not a one-time task — it is a continuous process that runs in parallel with your experiment cycle.

Phases of literature work

1. **Orientation**: understand the field broadly (2–3 survey papers)
2. **Depth**: read the key papers in your specific area
3. **Tracking**: follow what cites key papers (forward) and what they cite (backward)
4. **Maintenance**: stay current (weekly arXiv digest)

Common mistake

Stopping at orientation. Most students read 5–10 papers and claim to know the state of the art. A publishable seminar paper requires knowing 20–40 papers in your specific area well enough to cite them accurately.

Understanding the related work is the key point, not necessarily *reading* all the papers. Use questions and seek for answers to gain understanding.

Where to Search

Primary databases

Database	Best for
Semantic Scholar	AI/ML, influence metrics, free
Google Scholar	broad, picks up preprints
ACM Digital Library	CS, IR, HCI (full text via university)
IEEE Xplore	signals, systems, AI
arXiv	preprints in cs.AI, cs.IR, cs.CL, cs.LG

Specialised

- Papers with Code — benchmarks + code
- Connected Papers — visual citation graph
- Litmaps — temporal citation maps

Search strategy

1. Start with a survey paper in your area
2. Extract key terms from it: methods, datasets, task names
3. Search those terms + "survey" or "benchmark"
4. Use Boolean: `query reformulation AND (dense retrieval OR BM25)`
5. Filter by year: focus on last 3–5 years, but include founding papers

Evaluating a source

- venue tier (A* or A conference = higher bar than workshop)
- citation count relative to age
- reproducibility: is code available?
- institution: is this an established research group?

Do not mistake preprint count for quality.

The Three-Pass Reading Method

Reading a paper ≠ reading it start to finish.

Pass 1 — 5 min
Title · abstract · intro (first + last ¶) · conclusion · skim headings & figures.
→ *triage: discard, pass 2, or pass 2 + 3*

Pass 2 — 20–30 min
Intro + related work · figures & tables · method headings + key equations · skip proofs.
→ *grasp the contribution; decide if it fits your gap*

Pass 3 — 60–90 min
Read critically · reconstruct the method · question every choice · note limitations.

Reading log entry *(after pass 3)*
[Author, Year]: Claims X. Evidence Y. Limitation Z. Relevant because W.

When to do pass 3
Surveys → pass 2 · related work → pass 1–2 · core papers in your scope → pass 3.

Forward and Backward Citation Tracking

The citation graph is your map. Traverse it deliberately.

Backward tracking (references)

Start from a core paper and follow its references:

- who introduced the problem?
- what methods does it build on?
- what datasets are standard?
- what are the founding works everyone cites?

Practical: read the related work section carefully — it is a curated introduction to the sub-field.

How deep to go: stop when you see the same papers appearing repeatedly (saturation).

Forward tracking (cited by)

Find papers that cite your core paper:

- who applied or extended this method?
- who criticised it?
- what newer systems replace it?

Tools:

- Semantic Scholar: "Highly Influential Citations" tab
- Google Scholar: "Cited by N" link
- Connected Papers: visual graph around one paper

Practical: for a seminar project, forward-track your 3–5 most central papers. This reveals the current frontier — papers from the last 12 months that build on your chosen method.

Managing Your Literature

An unorganised reading list is a forgotten reading list.

Tool stack (recommended)

Tool	Role
Zotero	reference manager · BibTeX export · browser plugin
Zotero PDF reader	annotate · highlight · link notes to page
Obsidian	link notes · build a literature map
arXiv RSS	weekly digest of new papers in your area

Zotero workflow

1. Browser plugin: one-click save from Google Scholar / ACM DL
2. Add paper to collection (one per project)
3. Read + annotate in Zotero PDF reader
4. Write reading log entry in Obsidian note
5. Link note to related notes (same method / dataset / claim)

Naming and tagging

In Zotero, tag papers with:

- `#core` — papers you cite directly
- `#baseline` — papers you compare to
- `#dataset` — dataset papers
- `#survey` — survey papers

BibTeX integration

- Zotero → Better BibTeX plugin → auto-export `.bib` file
- Point LaTeX `\bibliography{}` at the exported file
- Keys are auto-generated: `Smith2023QueryReformulation`

Anti-pattern: a flat folder called "papers" with 200 PDFs named "paper1.pdf". This is not a literature system.

Identifying the Gap from Literature

After reading, you can map the field — and locate what is missing.

Literature map structure

Draw (physically or in Obsidian) a map of:

1. **Task** — what problem is being solved?
2. **Dominant methods** — what approaches exist?
3. **Datasets & benchmarks** — what is the standard evaluation?
4. **Open questions** — where do papers disagree or stop?

Your gap lives in quadrant 4.

Example for RAG on Open Web

- Task: open-domain QA with web retrieval
- Methods: BM25, dense retrieval, hybrid, re-ranking
- Benchmarks: NQ, TriviaQA, MSMARCO, BEIR
- Open: no standard evaluation on open-crawl corpora; limited agent-driven retrieval studies

Gap quality checklist

- It is not already addressed by a paper you missed
- It is bounded enough to study in 10–15 weeks
- It connects to your available infrastructure (OWI)
- A result in either direction is interesting
- You can name a concrete evaluation protocol

From gap to research question

> *Gap: "No study compares sparse vs. dense retrieval on open-crawl corpora for multi-hop QA."*

> *Research question: "Does hybrid BM25+dense retrieval outperform BM25-only on multi-hop OWI queries, measured by Recall@10?"*

A gap that you can state precisely in two sentences is probably real. A gap that requires five sentences of qualification is probably not.

Scientific Publishing

Guiding Question: How does a paper get from submission to publication — and what do reviewers actually look for?

Scientific Publishing — Overview

Module Overview

What we cover

- the anatomy of a research paper
- the peer review process
- where to publish in AI/ML/IR
- writing process and common rejection reasons

Why it matters for you

Understanding what reviewers look for will make your report better — even though this seminar report is not submitted to a venue.

Anatomy of a Research Paper

Standard structure

Section	Purpose	Length
Abstract	claim + evidence + impact	150–200 words
Introduction	motivation, gap, contribution, preview	1–1.5 pages
Related Work	what exists, why it is not enough	1 page
Method	what you did and why	1–x pages
Experiments	setup, data, baselines, metrics	0.5–x page
Results	numbers, tables, figures	1–x pages
Analysis	what it means, limits, surprising findings	0.5–x page
Conclusion	restate claim, future work	0.5 page

Writing order (recommended)

1. Method section first — clarifies your thinking
2. Experiments section — what you actually did
3. Results — tables and figures
4. Analysis — what it means
5. Related Work — once you know your contribution
6. Introduction — once you know the whole story
7. Abstract — last, distils everything

Figures and tables: Every figure needs a caption that states the takeaway, not just what is shown:

> *Bad: "Figure 2: nDCG@10 results."*

> *Good: "Figure 2: Hybrid retrieval outperforms BM25 on multi-hop queries but not single-hop, suggesting query complexity moderates the retrieval advantage."*

The Peer Review Process

Source: Derntl 2014

How it works

1. Authors submit paper to a venue by the deadline
2. Programme Chairs assign papers to Area Chairs (ACs)
3. ACs recruit 3–4 reviewers (Programme Committee, PC)
4. Reviewers write confidential reviews (4–6 weeks)
5. Authors write a rebuttal (2–3 days)
6. Reviewers and AC discuss (1–2 weeks)
7. AC makes a recommendation
8. Programme Chairs make final decisions
9. Notification sent to authors

Timeline: 3–5 months from submission to notification

Double-blind: at most venues, reviewers do not know who the authors are (and authors do not see reviewer names)

What reviewers look for

Criterion	Weight
Novelty	Is the contribution new?
Significance	Does it matter?
Correctness	Are the experiments valid?
Clarity	Is the paper well-written?
Reproducibility	Can others repeat this?

Review score scale (typical)

- strong accept
- weak accept
- borderline (most common)
- reject

Borderline papers are the most common. The rebuttal matters for them — address reviewer concerns directly and concisely.

Where to Publish in AI/ML/IR

Top venues by area

Area	Conference	Notes
ML (general)	NeurIPS, ICML, ICLR	very competitive
NLP / LLMs	ACL, EMNLP, NAACL	linguistics + ML
IR / Search	SIGIR, ECIR, WSDM	ranking + retrieval
AI (general)	AAAI, IJCAI	broad
Data Mining	KDD, WWW	scale + web
Human factors	ACM CHI	user studies
Workshops	at any of the above	lower bar, good for first paper

Journal alternatives

- JMLR (open access, fast)
- ACM TOIS (IR)
- AIJ (Artificial Intelligence Journal)

How to choose a venue

1. Where do the papers you cite appear most often?
2. Is the timeline compatible with your work?
3. Is the scope a good match?
4. Is the acceptance rate reasonable given the paper's maturity?

Acceptance rates (approximate)

Venue	Rate
NeurIPS	15–26%
ICML	17–26%
SIGIR (long)	20–25%
ECIR (long)	22–28%
Workshops	40–60%

Common Rejection Reasons

Reviewers reject papers for predictable reasons. Most can be avoided.

Scientific reasons

- Weak or absent baseline: "Compared only to a 2-year-old system"
- No statistical significance: "Results may be noise"
- Data leakage or experimental flaw: "Evaluation is invalid"
- Overclaiming: "The authors state X but their experiment only shows Y"
- Cherry-picking: "Only Table 2 is discussed; Tables 3 and 4 show the approach fails"
- Irreproducibility: "No code, no data, insufficient detail to reimplement"

Writing and framing reasons

- Unclear contribution: "It is not obvious what is new here"
- Missing related work: "The authors seem unaware of [key paper]"
- Unclear motivation: "Why does this problem matter?"
- Poor structure: "Section 3 contradicts Section 2"
- Abstract does not match the paper
- Results do not support the claimed conclusion

Anti-rejection checklist

- State the contribution explicitly in the introduction
- Cite the most relevant 3 papers to your exact method
- Use a strong, recent baseline
- Run significance tests
- Make code and data available

Project Management with GTD and Git

Guiding Question: How do you stay on top of a long, non-linear research project without losing track of decisions and results?

Project Management — Overview

Module Overview

Research is a long, non-linear process. Without a system, tasks accumulate, experiments are forgotten, and the writing phase becomes chaotic.

Two complementary systems

- **GTD (Getting Things Done)**: manage what you need to do
- **Git**: manage what you have done and what exists

Scope for this seminar

You are managing one project over 5 months. The system does not need to be elaborate — it needs to be consistent.

GTD for Researchers

GTD (Allen 2001): get it out of your head, into a trusted system.

The five steps

Step	Action
Capture	write everything down
Clarify	what's the next physical action?
Organise	right list or reference
Reflect	weekly review
Engage	pick and do

Mapped to research

GTD	Research
Inbox	ideas, findings, comments
Next Actions	"run pilot on 1% data"
Projects	seminar paper
Waiting For	supervisor feedback
Someday	follow-up RQs
Reference	lit notes, exp logs

Minimal tool: one text file — Inbox · Next Actions · Waiting For · Someday · Reference.

10-minute weekly review prevents the "what did I decide three weeks ago?" problem.

GTD in Practice: The Research Week

Daily (10 min)

- Morning: pick 1–3 next actions
- Evening: capture + log results

Weekly review (30 min)

1. Empty inbox
2. Update next actions
3. Follow up on waiting-for
4. Review experiment log
5. Set 3 actions for next week

Deep work

90-minute uninterrupted blocks. No email, no Slack.

Typical capture items

- *Idea:* try a different chunking strategy
- *Read:* Chen2024 — relevant to RQ3
- *Todo:* verify test-set hold-out
- *Question:* is 3 seeds enough?
- *Finding:* BM25 beats dense on short queries

Reference, not actions

Literature notes · experiment results · design decisions · commit links.

Anti-pattern: inbox as reference. A full inbox = failed system.

Git for Research Projects

Git is not just version control for code — it is an audit trail for your scientific decisions.

What to track in Git

- all experiment code
- configuration files (hyperparameters, prompts, data splits)
- evaluation scripts
- analysis notebooks
- paper source (LaTeX)
- result logs (CSV or JSON, not binary model weights)

Commit discipline

- commit after each meaningful change
- commit message = what changed + why
- never commit "fix" or "update" without detail

Example commit messages

```
Add BM25 baseline with Lucene 9 · nDCG@10 = 0.312
Switch to sentence-transformers/e5-large for dense retrieval
Fix data split: test queries were leaking into dev set
Add Wilcoxon test for BM25 vs dense comparison
```

Every commit is a decision documented.

Branching strategy for ML experiments

```
main          ← stable, reproducible baseline
 │
 ├─── exp/bm25-baseline
 ├─── exp/dense-e5
 ├─── exp/hybrid-v1
 └─── exp/hybrid-v2-reranked
```

- one branch per experimental condition
- merge to main only when results are confirmed
- tag the commit that produced your reported results

Results tracking

Avoid manual copy-paste of results. Use:

- MLflow (local) or Weights & Biases (cloud) for run tracking
- JSON/CSV logs committed to Git after each run
- A `results/` directory with one file per experiment condition

Never hardcode paths. Use config files or environment variables.

Repository Structure for ML Experiments

Recommended structure

```
project/
├── data/
│   ├── raw/            ← original, never modified
│   ├── processed/      ← reproducibly derived from raw
│   └── splits/         ← fixed train/dev/test splits
├── src/
│   ├── retrieval/
│   ├── generation/
│   └── evaluation/
├── experiments/
│   ├── configs/        ← YAML/JSON per run
│   └── results/        ← JSON/CSV output
├── notebooks/
│   └── analysis/       ← EDA, result visualisation
├── paper/
│   └── main.tex
├── README.md
└── requirements.txt  (or pyproject.toml)
```

Key principles

Raw data is read-only: Never modify `data/raw/`. All processing is scripted so it is reproducible.

Configs are code: Every experiment is defined by a config file. The same config + code = same result.

Notebooks are for exploration only: Do not put core logic in notebooks. Export final analysis to scripts.

README is a contract: The README should state: (i) what the project does, (ii) how to reproduce the main result, (iii)where results are stored

Data too large for Git: Use Git LFS for large files, or store in object storage (S3, GCS) and reference the path in the config.

Toooling: Use proper tooling and develop your own Command Line Interface (CLI) to orchestrate your experiments.

A new collaborator (or future-you) should be able to reproduce your main result in under 30

The Research Diary

A research diary is the private record of what you tried, what failed, and why you made each decision.

What to write

Entry type	Example
Decision log	"Chose E5-large over BGE because E5 has better multilingual coverage"
Failure log	"Run 3 crashed — NaN loss after epoch 2. Found: learning rate too high"
Finding log	"BM25 unexpectedly beats dense on short queries — check token coverage"
Idea log	"What if we use query expansion before dense retrieval?"
Literature note	"Smith2023 §3: they use a 90/10 dev/test split — different from our setup"

Minimum viable diary

One Markdown file per week: `2026-04-21-week-diary.md`
Add a timestamped bullet for every meaningful event.

Why the diary matters

During the project:

- prevents repeating experiments you already ran
- captures insights that would otherwise be lost
- makes the weekly review fast

During writing:

- related work: your reading notes become citations
- method: your design decisions become the rationale
- analysis: your failure log becomes the "we investigated X" paragraph
- limitations: your idea log becomes future work

For your supervisor / reviewers:

- shows your reasoning process
- demonstrates scientific rigour
- provides traceability

> The paper is the tip of the iceberg. The diary is everything below the surface that makes the paper credible.

Wrap-Up

Your Scientific Infrastructure

Literature

- search: Semantic Scholar + Google Scholar + arXiv
- read: three-pass method
- navigate: backward + forward citation tracking
- organise: Zotero + Better BibTeX + Obsidian notes
- find the gap: literature map, four-quadrant analysis

Publishing

- anatomy: abstract → intro → related → method → experiments → results → analysis → conclusion
- process: submit → review → rebuttal → decision
- where: match venue to area, target workshops first
- avoid: weak baselines, missing significance, overclaiming

> You do not need the perfect system. You need a consistent system that you actually use.

Project Management

- GTD: capture → clarify → organise → reflect → engage
- Weekly review: 30 minutes keeps everything on track
- Git: one commit per decision, meaningful messages
- Repository: raw data read-only, configs are code, README is a contract
- Research diary: log decisions, failures, and findings

Your toolbox

Need	Tool
References	Zotero + Better BibTeX
Notes / map	Obsidian
Code	Git + GitHub/GitLab
Experiments	MLflow or W&B
Paper	LaTeX + ACM format
Task management	plain text + weekly review

Image Credits & References

Allen, David (2001). Getting Things Done: The Art of Stress-Free Productivity. *Viking*.

Derntl, Michael (2014). Basics of Research Paper Writing and Publishing. *International Journal of Technology Enhanced Learning*.